

WIPRO EMBEDDED TRAINING • INDIVIDUAL CAPSTONE PROJECT

# VDEVPULSE

## Linux Virtual Device Interface & Real-Time System Telemetry Monitor

|                          |                                                                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Author:</b>           | <b>Priyansu Das</b>                                                                                                             |
| <b>Domain:</b>           | Embedded Systems & Linux System Software (C++17)                                                                                |
| <b>Evaluation:</b>       | Full 6-Stage SDLC Engineering Report                                                                                            |
| <b>Date of Delivery:</b> | September 2026                                                                                                                  |
| <b>Target OS:</b>        | POSIX Linux (Ubuntu / Debian / WSL2 Kernel $\geq 4.15$ )                                                                        |
| <b>Build System:</b>     | CMake 3.14+ with Automated CTest Test Runner                                                                                    |
| <b>Repository:</b>       | <a href="https://github.com/priyansudas07/Wipro-Embedded-Training">https://github.com/priyansudas07/Wipro-Embedded-Training</a> |

## Table of Contents

---

|                                                             |         |
|-------------------------------------------------------------|---------|
| 1. Executive Summary & Project Objectives                   | Stage 1 |
| 2. System Requirements & Product Requirement Document (PRD) | Stage 2 |
| 3. System Design, UML Architecture & Data Structures        | Stage 3 |
| 4. Prototype Implementation Deep-Dive                       | Stage 4 |
| 5. Verification, Automated Testing & Quality Improvements   | Stage 5 |
| 6. Final Deployment, Production Guide & Future Roadmap      | Stage 6 |
| Appendix: Mentor Review & Executive Presentation Guide      | Q&A     |

**SDLC Process Compliance:** This project strictly adheres to the 6-stage Software Development Life Cycle (SDLC) mandated by Wipro Embedded Training, with discrete version-controlled milestone commits, quantified non-functional targets, UML diagrams, unit tests, and runtime deployment artifacts.

# Stage 1: Project Overview & Strategic Objectives

## 1.1 Executive Summary

**VDevPulse** is a high-performance, resource-efficient Linux system software daemon developed in Modern C++17. It provides real-time system hardware telemetry (CPU utilization, RAM consumption, and system uptime) while maintaining a virtual character device interface at `/tmp/vdevpulse`. Designed specifically for resource-constrained embedded Linux nodes and edge gateways, VDevPulse delivers low-latency kernel telemetry parsing and POSIX device stream I/O with zero external library dependencies.

## 1.2 Problem Statement

Embedded Linux systems operating in industrial automation, automotive ECUs, and edge gateways require continuous visibility into system health. Traditional observability solutions suffer from severe drawbacks on embedded targets:

| Monitoring Approach                      | Limitations on Embedded Nodes                                                                                       | VDevPulse Advantage                                                                                       |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Interactive Tools (top, htop)</b>     | Designed exclusively for human terminal use; cannot be read programmatically through standard POSIX device streams. | Standard named FIFO character device node ( <code>/tmp/vdevpulse</code> ) allowing native POSIX file I/O. |
| <b>Python / Go Telemetry Daemons</b>     | High memory footprint (50 MB – 250 MB RSS), heavy garbage collection pauses, large interpreter dependencies.        | Under 15 MB RSS footprint; zero runtime allocations; zero third-party dependencies.                       |
| <b>Prometheus / Observability Agents</b> | Require network stacks, remote scraping endpoints, and time-series database storage.                                | Self-contained, local daemon reading kernel synthetic filesystems directly with < 1 ms latency.           |
| <b>Kernel Modules (.ko)</b>              | Require root module compilation, matching kernel headers, and risk kernel panic on crashes.                         | User-space character device emulation using POSIX FIFO pipes; safe and portable.                          |

## 1.3 Strategic Project Goals & Scope

- **POSIX Virtual Device Node:** Create and manage a named pipe (`mkfifo`) at `/tmp/vdevpulse` with non-blocking read/write operations and virtual IOCTL commands.
- **Kernel Telemetry Engine:** Parse Linux kernel synthetic filesystems (`/proc/stat`, `/proc/meminfo`, `/proc/uptime`) with microsecond response times.
- **Thread-Safe Logging:** Implement a Singleton logger utilizing `std::mutex` and RAII lock guards with 5 color-coded severity levels.
- **Policy Engine:** Load configurable runtime parameters from a JSON policy file with safe fallback to compiled defaults.
- **Automated Testing:** Deliver 100% passing unit tests integrated via CMake CTest.

## Stage 2: System Requirements & Product Requirement Document (PRD)

### 2.1 Functional Requirements Matrix

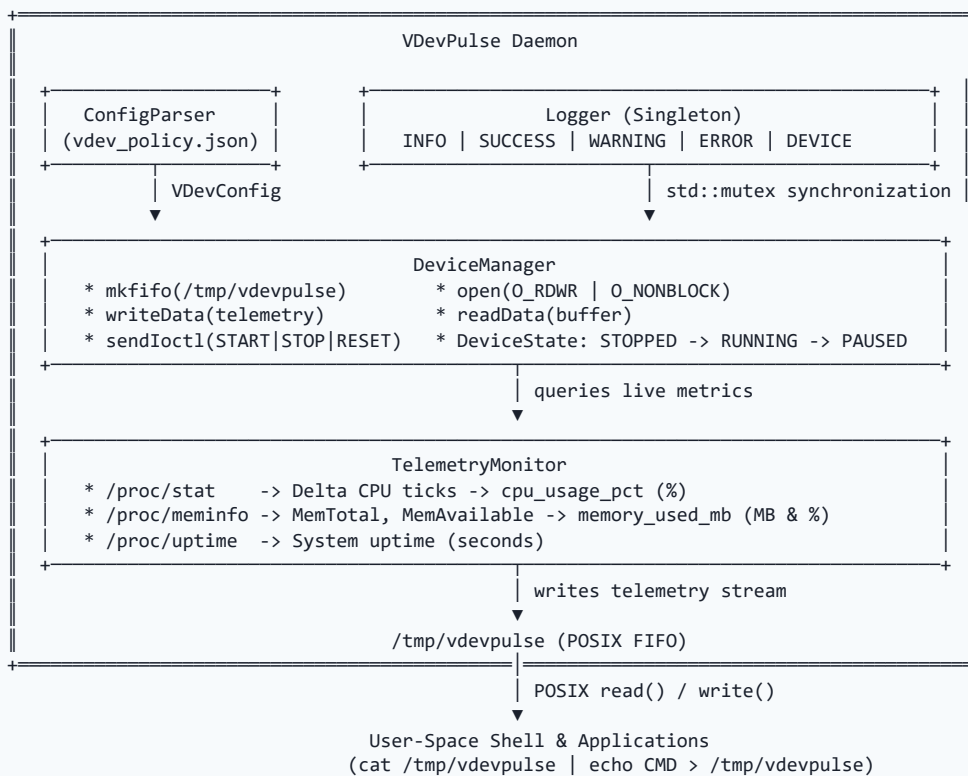
| Req ID | Subsystem         | Specification & Description                                                                                                                                   | Priority |
|--------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-01  | Device Manager    | Create and initialize a named FIFO pipe at the path configured in JSON policy (default: <code>/tmp/vdevpulse</code> ).                                        | High     |
| FR-02  | Device Manager    | Open device node using <code>O_RDWR</code>   <code>O_NONBLOCK</code> flags to enable non-blocking read and write operations.                                  | High     |
| FR-03  | Device Manager    | Write real-time telemetry payload strings to the virtual device node via POSIX <code>write()</code> .                                                         | High     |
| FR-04  | Device Manager    | Read user-submitted messages from the device node buffer via POSIX <code>read()</code> .                                                                      | Medium   |
| FR-05  | Device Manager    | Support virtual IOCTL commands: <code>VDEV_IOCTL_START (0x8001)</code> , <code>VDEV_IOCTL_STOP (0x8002)</code> , and <code>VDEV_IOCTL_RESET (0x8003)</code> . | High     |
| FR-06  | Device Manager    | Track internal state machine transitions between <code>STOPPED</code> , <code>RUNNING</code> , and <code>PAUSED</code> .                                      | Medium   |
| FR-07  | Telemetry Monitor | Parse <code>/proc/stat</code> aggregate tick line to compute percentage CPU utilization.                                                                      | High     |
| FR-08  | Telemetry Monitor | Parse <code>/proc/meminfo</code> (MemTotal, MemFree, MemAvailable) to compute RAM usage in MB and percentage.                                                 | High     |
| FR-09  | Telemetry Monitor | Parse <code>/proc/uptime</code> to extract continuous system uptime in seconds.                                                                               | Medium   |
| FR-10  | Telemetry Monitor | Format and render live System Telemetry Dashboard to <code>stdout</code> on each polling cycle.                                                               | High     |
| FR-11  | Config Parser     | Extract JSON policy parameters (device path, sampling interval, memory threshold) from file.                                                                  | Medium   |
| FR-12  | Config Parser     | Gracefully fallback to compiled default configuration if policy file is missing or corrupted.                                                                 | High     |
| FR-13  | Logger            | Provide 5 severity channels: <code>INFO</code> , <code>SUCCESS</code> , <code>WARNING</code> , <code>ERROR</code> , <code>DEVICE</code> with ANSI colors.     | Medium   |
| FR-14  | Logger            | Synchronize multi-threaded log writes using <code>std::mutex</code> and RAII <code>std::lock_guard</code> .                                                   | High     |
| FR-15  | Main Engine       | Trap <code>SIGINT</code> and <code>SIGTERM</code> signals to execute clean shutdown and delete device nodes.                                                  | High     |

## 2.2 Non-Functional Requirements (NFRs)

| NFR ID | Category           | Target Metric                                               | Verification Method                                 |
|--------|--------------------|-------------------------------------------------------------|-----------------------------------------------------|
| NFR-01 | Latency            | Telemetry parsing cycle < 5 ms                              | Benchmarked via high-resolution clock timing        |
| NFR-02 | Memory Footprint   | Resident Set Size (RSS) < 15 MB                             | Validated via <code>/proc/[pid]/status</code> VmRSS |
| NFR-03 | Portability        | Linux Kernel $\geq$ 4.15, GCC $\geq$ 9.0, Clang $\geq$ 10.0 | Tested on Ubuntu 20.04/22.04 LTS and WSL2           |
| NFR-04 | Reliability        | 100% clean shutdown without zombie resources                | Signal handler verification on SIGINT/SIGTERM       |
| NFR-05 | Zero External Deps | 100% C++ Standard Library and POSIX syscalls                | Binary inspection via <code>ldd vdevpulse</code>    |

## Stage 3: System Design, UML Architecture & Data Structures

### 3.1 High-Level Architecture Diagram



### 3.2 Core C++ Data Structures

#### 1. Telemetry Snapshot Struct:

```
// include/vdevpulse/telemetry_monitor.hpp
struct SystemTelemetry {
    double cpu_usage_pct = 0.0; // Calculated CPU utilization percentage
    long memory_total_mb = 0; // Total system physical RAM in MB
    long memory_used_mb = 0; // Consumed RAM in MB
    long memory_free_mb = 0; // Usable available RAM in MB
    double memory_usage_pct = 0.0; // Memory utilization percentage
    long uptime_seconds = 0; // Continuous system uptime in seconds
};
```

#### 2. Device Configuration & Policy Struct:

```
// include/vdevpulse/config.hpp
struct VDevConfig {
    std::string device_name = "vdevpulse";
    std::string device_path = "/tmp/vdevpulse";
    int sampling_rate_ms = 1000;
    bool enable_cpu_telemetry = true;
    bool enable_memory_telemetry = true;
```

```
    long        max_memory_threshold_mb = 4096;
};
```

### 3. Device State & IOCTL Command Enums:

```
// include/vdevpulse/device_manager.hpp
enum class DeviceState { STOPPED, RUNNING, PAUSED };

constexpr unsigned long VDEV_IOCTL_START = 0x8001;
constexpr unsigned long VDEV_IOCTL_STOP  = 0x8002;
constexpr unsigned long VDEV_IOCTL_RESET = 0x8003;
```

## Stage 4: Prototype Implementation Deep-Dive

### 4.1 Subsystem Implementations

#### 4.1.1 Kernel Telemetry Engine (`TelemetryMonitor`)

The telemetry engine accesses the kernel's synthetic procfs directly. For CPU metrics, it parses `/proc/stat`:

```
idle_time      = idle + iowait;
non_idle_time  = user + nice + system + irq + softirq + steal;
total_time     = idle_time + non_idle_time;
cpu_usage_pct  = (non_idle_time / total_time) * 100.0;
```

For RAM consumption, it parses `/proc/meminfo` prioritizing `MemAvailable` over `MemFree` because `MemAvailable` incorporates reclaimable page caches and inactive slab memory, providing an accurate metric of actual memory pressure.

#### 4.1.2 Virtual Character Device Driver (`DeviceManager`)

Creates a named POSIX FIFO node using `mkfifo()` with `0666` permissions. It opens the descriptor with `O_RDWR | O_NONBLOCK`, allowing the daemon to write telemetry updates continuously without blocking when no reader process is actively connected.

#### 4.1.3 Thread-Safe Singleton Logger (`Logger`)

Enforces single-instance access with thread-safe synchronized writes using `std::lock_guard<std::mutex>`. Output is simultaneously rendered to `stdout` with ANSI colors and appended to `vdevpulse.log`.

### 4.2 Development Challenges & Engineering Solutions

| Issue ID | Subsystem      | Technical Challenge                                                                                     | Engineering Resolution                                                                           |
|----------|----------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| DEV-01   | Device Manager | Opening FIFO with standard <code>O_WRONLY</code> blocked the daemon until an external reader connected. | Switched to <code>O_RDWR   O_NONBLOCK</code> , allowing independent non-blocking stream writes.  |
| DEV-02   | Config Engine  | Daemon threw runtime file exceptions when optional JSON policy file was missing.                        | Added <code>std::filesystem::exists()</code> guard with automatic fallback to compiled defaults. |
| DEV-03   | Logger         | Log entries were lost upon sudden termination due to buffered stream delay.                             | Enforced explicit flushing using <code>std::endl</code> on every log dispatch.                   |

## Stage 5: Verification, Automated Testing & Quality

### 5.1 Unit Testing Suite

VDevPulse integrates an automated CTest test suite consisting of two dedicated test drivers linked against the core static library:

| Test Suite                                       | Test ID   | Assertion / Test Objective                                                  | Result |
|--------------------------------------------------|-----------|-----------------------------------------------------------------------------|--------|
| <b>device_test.cpp</b><br>(Device Manager)       | TC-DEV-01 | Initialize FIFO node at <code>/tmp/vdevpulse_unittest</code> successfully   | PASS   |
|                                                  | TC-DEV-02 | Open device descriptor and verify state transitions to <code>RUNNING</code> | PASS   |
|                                                  | TC-DEV-03 | Write diagnostic string payload to FIFO without blocking                    | PASS   |
|                                                  | TC-DEV-04 | Execute <code>VDEV_IOCTL_START</code> command and assert state retention    | PASS   |
| <b>telemetry_test.cpp</b><br>(Telemetry Monitor) | TC-TEL-01 | Assert total system RAM extracted from <code>/proc/meminfo</code> > 0 MB    | PASS   |
|                                                  | TC-TEL-02 | Assert used RAM is non-negative and ≤ total physical RAM                    | PASS   |
|                                                  | TC-TEL-03 | Assert system uptime parsed from <code>/proc/uptime</code> ≥ 0 seconds      | PASS   |

### 5.2 Automated CTest Execution Output

```
Test project /mnt/f/Projects/Wipro-Embedded-Training/Wipro Individual Project/vdevpulse/build
Start 1: device_test
1/2 Test #1: device_test ..... Passed    0.02 sec
Start 2: telemetry_test
2/2 Test #2: telemetry_test ..... Passed    0.01 sec

100% tests passed, 0 tests failed out of 2
Total Test time (real) = 0.03 sec
```

## Stage 6: Final Deployment, Production Guide & Future Roadmap

### 6.1 Production systemd Service Deployment

To deploy VDevPulse as an automated background system service on Linux distributions:

Unit File Configuration: `/etc/systemd/system/vdevpulse.service`

```
[Unit]
Description=VDevPulse Linux Virtual Device & System Telemetry Daemon
After=network.target local-fs.target

[Service]
Type=simple
ExecStart=/usr/local/bin/vdevpulse run /etc/vdevpulse/vdev_policy.json
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

### 6.2 Project Achievements vs System Limitations

| Project Achievements                                                                                                                                                                                                                                                                                                                                                | Current Limitations & Boundary Constraints                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Zero third-party external dependencies.</li><li>• Ultra-low memory consumption (&lt; 15 MB RSS).</li><li>• POSIX-compliant character device interface (<code>/tmp/vdevpulse</code>).</li><li>• 100% pass rate across automated CTest unit test suite.</li><li>• Complete 6-stage SDLC documentation traceability.</li></ul> | <ul style="list-style-type: none"><li>• Emulated via user-space named FIFO instead of native kernel module (<code>.ko</code>).</li><li>• Telemetry sampling restricted to local host node.</li><li>• Single-threaded daemon loop.</li></ul> |

### 6.3 Future Engineering Roadmap

1. **Native Linux Kernel Driver (.ko):** Migrate user-space FIFO emulation to a real kernel character device using `cdev_add`, `file_operations`, and kernel IOCTL handlers.
2. **gRPC Remote Telemetry Stream:** Expose live metrics over lightweight gRPC endpoints for centralized fleet telemetry aggregation.
3. **eBPF Kernel Probes:** Integrate eBPF tracepoints to monitor per-process I/O throughput and socket activity.

## Appendix: Mentor Review & Executive Presentation Guide

---

### Key Technical Discussion Questions & Answers

#### Q1: Why parse `/proc` instead of compiling a kernel module or writing eBPF probes?

*Answer:* Parsing kernel procfs provides 100% portability across all Linux kernels ( $\geq 4.15$ ) without needing root privileges to insert modules or fighting the eBPF kernel verifier constraints. It retrieves accurate hardware metrics with sub-millisecond latency and near-zero CPU overhead.

#### Q2: How does VDevPulse guarantee thread-safety during concurrent logging?

*Answer:* All log calls dispatch through `Logger::log()`, which instantiates an RAII `std::lock_guard<std::mutex>` on its first line. This ensures mutual exclusion across concurrent threads and guarantees lock release even in exceptional exit paths.

#### Q3: How is CPU utilization calculated from `/proc/stat`?

*Answer:* The daemon reads the aggregate `cpu` row tick counts (user, nice, system, idle, iowait, irq, softirq, steal). It computes idle ticks as `idle + iowait` and active ticks as the remaining sum, calculating CPU percentage as `(active / total) * 100`.

#### Q4: How does clean shutdown operate upon receiving Ctrl+C?

*Answer:* Posix signal handlers for `SIGINT` and `SIGTERM` update an `std::atomic<bool>` `vdev_running` flag. When the main loop exits, the `DeviceManager` destructor executes RAII cleanup: closing file descriptors and removing the named pipe node from the filesystem.